

VITALSENSE AI

Predicting Disease from Vitals & Symptoms

A supervised machine-learning pipeline that turns 9 vitals and 30 symptom flags into a ranked disease prediction across 15 conditions.

Random Forest Classifier · scikit-learn · 90.4% Test Accuracy

Model Training Pipeline Overview

Symptom checkers today guess with rules, not data

VitalSense AI's original front end is a JavaScript rule-engine: **hand-written if/else logic** mapping symptoms to a likely diagnosis.

Rule engines don't generalize.

They can't weigh 39 interacting features, don't improve with more data, and struggle where several diseases share symptoms (fever + fatigue + body pain shows up in Flu, Dengue, Malaria, Typhoid, and COVID-19 alike).

This project replaces that rule engine with a trained statistical model that learns patterns directly from labeled patient data.

15

overlapping disease classes to distinguish between

39

input features — 9 vitals + 30 binary symptoms

0

generalization in a hard-coded rule engine

The Dataset

4,000 synthetic patient records, fully labeled

4,000 patient rows generated via generate_dataset.py	9 numeric vitals age, temp, BP, HR, sugar, O2...	30 binary symptoms fever, cough, fatigue, rash...	15 disease labels Cold, Flu, COVID-19, Dengue...
--	--	---	--

Split strategy

80 / 20 stratified train-test split (3,200 train / 800 test) preserves the class balance across all 15 diseases — plus a 5-fold cross-validation pass on the training set to sanity-check stability before touching the held-out test data.

Every feature the model sees

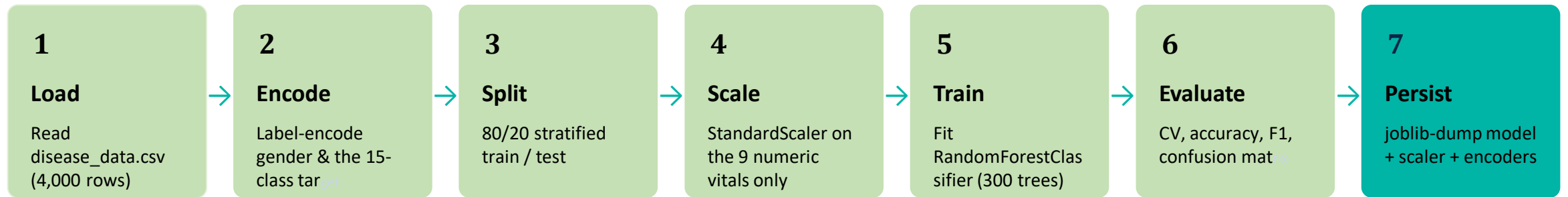
VITALS (numeric, 10)

- Age
- Gender
- Weight (kg) & Height (cm)
- Body temperature (°F)
- Blood pressure — systolic / diastolic
- Heart rate
- Blood sugar
- Oxygen saturation

SYMPTOMS (binary flags, 30)

- | | | |
|---------------|-----------------------|----------------------|
| • Fever | • Chills | • Anxiety |
| • Cough | • Runny nose | • Depression |
| • Headache | • Shortness of breath | • Back pain |
| • Sore throat | • Joint pain | • Ear pain |
| • Fatigue | • Skin rash | • Eye pain |
| • Chest pain | • Abdominal pain | • Swollen glands |
| • Vomiting | • Dizziness | • Frequent urination |
| • Diarrhea | • Loss of smell | • Weight loss |
| • Nausea | • Loss of taste | • Constipation |
| • Body pain | • Sweating | • Muscle weakness |

Training Pipeline



Only the 9 numeric vitals pass through StandardScaler — the 30 symptom flags are already 0/1 and need no scaling.

A tuned Random Forest classifier

Why a Random Forest

- Handles a mix of continuous vitals and binary symptom flags natively
- Captures non-linear, interacting symptom combinations without manual feature engineering
- Robust to noisy or weakly-informative symptom flags via bagging
- Feature importances give a built-in, interpretable audit trail

Hyperparameters

Estimator	RandomForestClassifier
n_estimators	300 trees
max_depth	14
min_samples_split	4
class_weight	balanced
random_state	42 (reproducible)
n_jobs	-1 (all cores)

Results

Consistently strong across validation methods

90.4%

Held-out test accuracy

0.903

Weighted F1 score

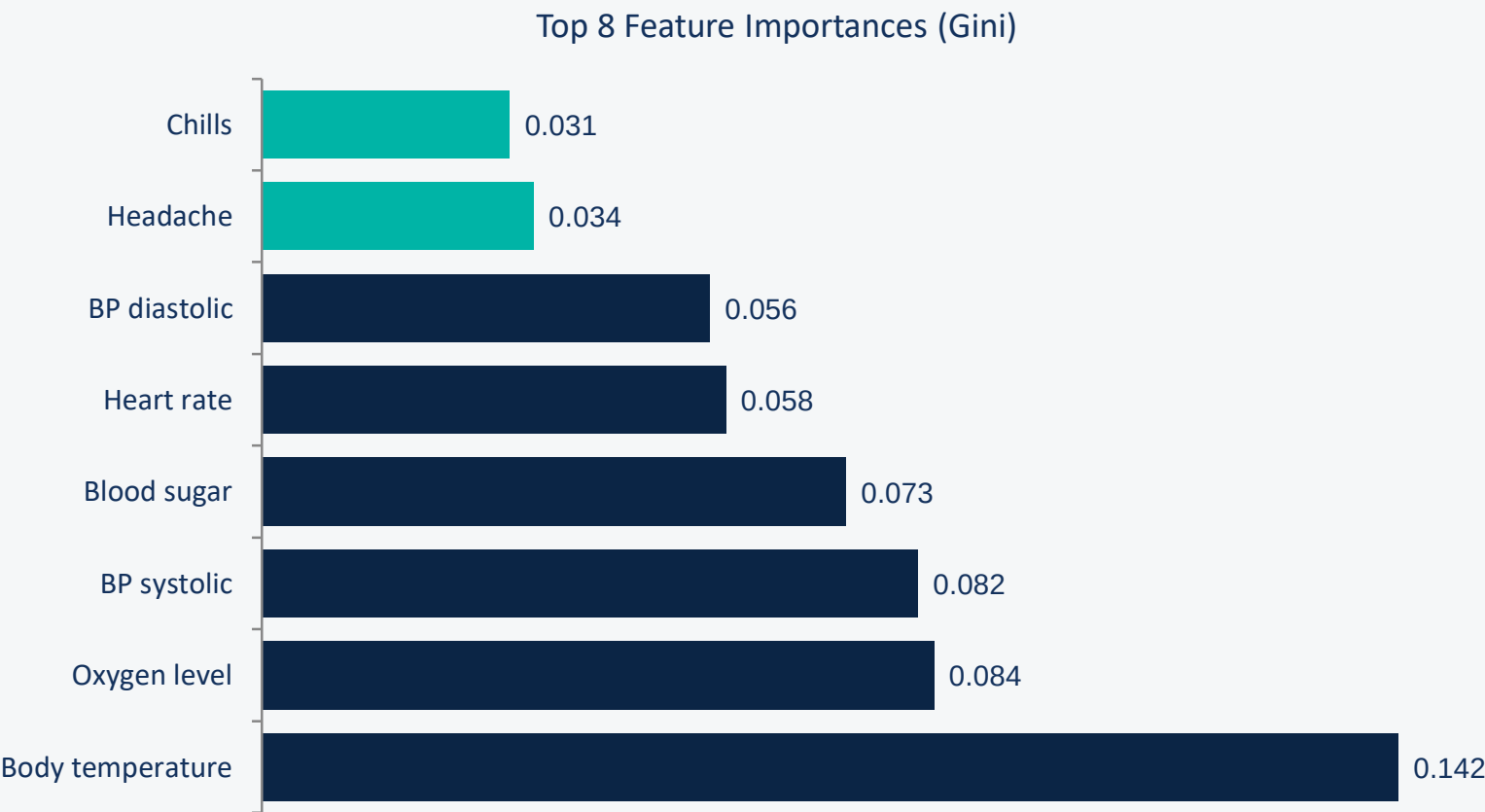
91.0%

5-fold CV mean accuracy

Model Performance



Vitals dominate the top of the importance ranking



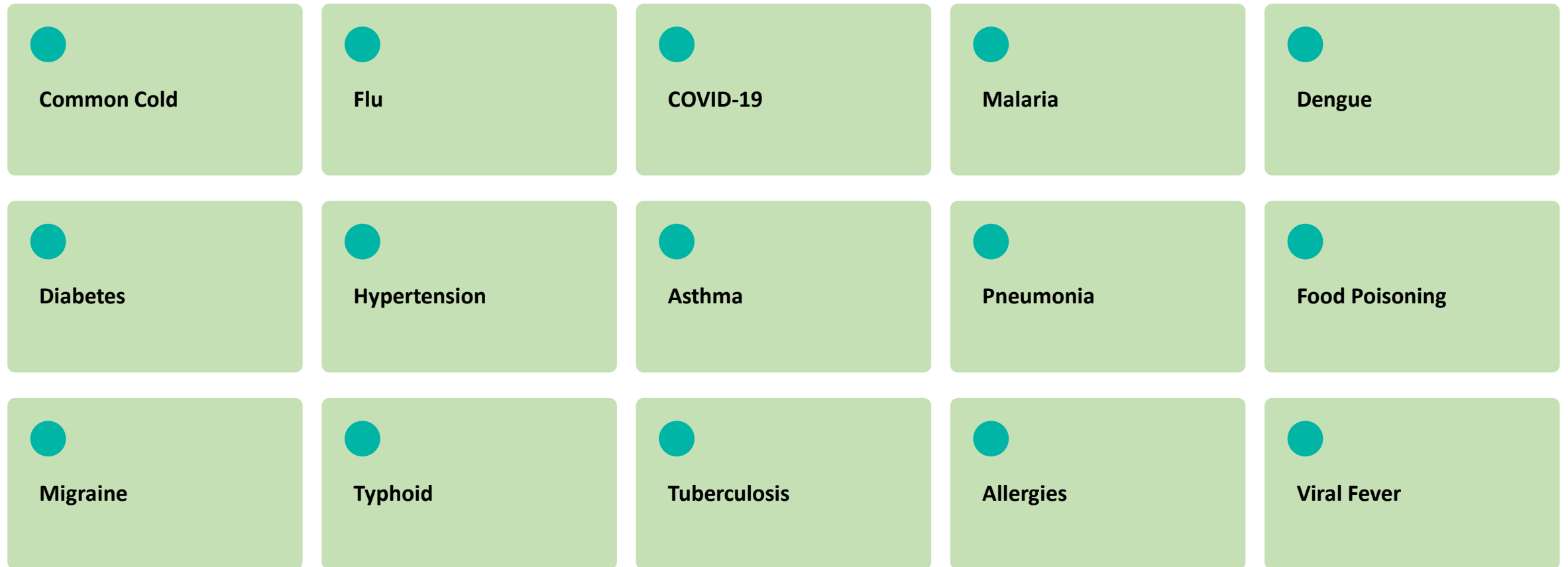
Reading it

The six vitals (navy) outrank every symptom flag individually — body temperature alone carries ~14% of total importance.

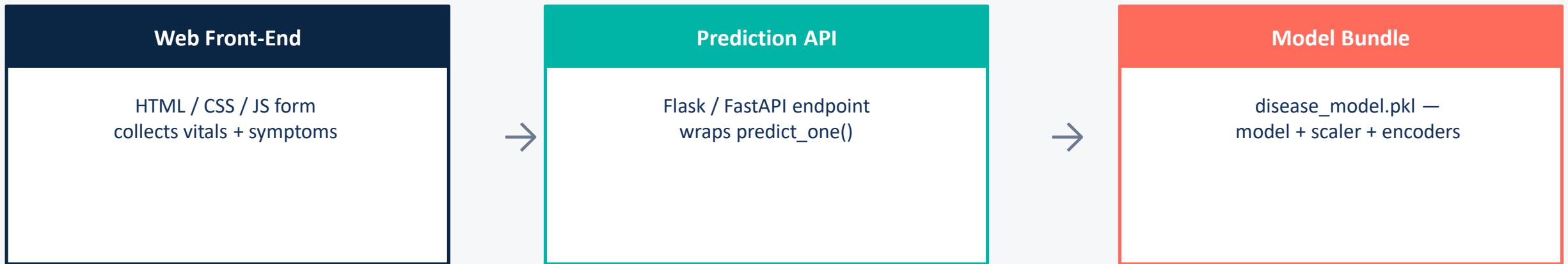
Symptoms still matter in combination

— they're what separates diseases that share similar vitals, like distinguishing Flu from Dengue.

15 disease classes, one shared feature space



From notebook model to live web app



Currently, the deployed VitalSense AI front end runs prediction logic client-side in JavaScript. Swapping in this trained model means replacing the local `runPrediction()` call with a `fetch()` to a `/predict` endpoint — no change needed on the form itself.

Next Steps

Where this pipeline goes from here

1

Scale the dataset

Regenerate with --rows 10000+
for more robust class coverage

2

Add real-world data

Validate against de-identified
clinical data under proper
governance

3

Serve via API

Wrap predict_one() in
Flask/FastAPI for the live web
front-end

4

Track drift

Re-run training_metrics.json
checks as the model is retrained
over time

Thank you — questions welcome